

Finding key edges in complex network through line graph transformation and deep reinforcement learning

Fengwei Guo, Chao Chen, Li Zeng, Tianle Pu, Feng Qing, Changjun Fan  *

College of Systems Engineering, National University of Defense Technology, Changsha, 410073, PR China

ARTICLE INFO

Keywords:

Complex network
Edge attack
Network dismantling
Line graph transformation
Deep reinforcement learning
Graph neural network

ABSTRACT

Finding high-value edge attack sequence in complex networks is essential for effectively deactivating harmful networks. However, prior works pay much attention to finding key nodes in complex networks, leading to a plethora of efficient methods for discovering critical nodes, while often ignoring the edges. Inspired by effective methods for finding key nodes, we ingeniously map the process of finding key edges to key node discovery through line graph transformation. Due to the inherently NP-hard nature of the network dismantling task, we design an end-to-end deep reinforcement learning framework to discover heuristics in a data-driven manner. This framework, named **FIGHTER** (Finding Key Edges in Complex Network through Line Graph Transformation and Deep Reinforcement Learning), treats the discovery of key edges as a learning task that can be modeled as a Markov Decision Process. We use the graph neural network to represent the complex network and employ deep reinforcement learning to uncover the underlying patterns necessary for finding key edges. Therefore, once FIGHTER has learned the heuristics from the training data, it can be repeatedly and efficiently applied to solve larger and more complicated networks. We design two network dismantling tasks: attacking the connected component with most nodes and most edges, respectively. According to the real-world scenarios, we consider situations where the cost of attacking edges is uniform and heterogeneous. Removal costs are equal to the degree of the line graph when the cost is heterogeneous. In these four experiments, compared with the best results obtained by the other five methods, FIGHTER has an average improvement of 20.45 % in the synthetic networks and 19.28 % in the real networks.

1. Introduction

Complex networks are commonly employed to model complex systems (Albert & Barabási, 2002; Boccaletti, Latora, Moreno, Chavez, & Hwang, 2006; Newman, 2003). Network dismantling (ND) (Shao, Huang, Stanley, & Havlin, 2014; Zdeborová, Zhang, & Zhou, 2016a) is a pivotal challenge in this field, which involves removing key nodes or edges to maximally degrade network functionality and effectively attacking harmful networks (Braunstein, Dall'Asta, Semerjian, & Zdeborová, 2016; Morone & Makse, 2015). Finding high-value nodes or edges holds wide-ranging applications across diverse scenarios, including drug design (Kuntz, 1992), epidemic control (Pastor-Satorras & Vespignani, 2001), and rumor management (Yiping, Gerald, Shlomo, Fredrik, & Stanley, 2008). However, due to their intrinsic NP-hard nature, these tasks cannot be solved by traditional algorithms within a reasonable time frame for large-scale networks (Lalou, Tahraoui, & Khedouci, 2018). Therefore, designing effective algorithms has garnered sig-

nificant interest and passion within the field over the past few decades (Artime et al., 2024a).

As a prospective topic, there are two primary strategies employed in network dismantling: node attack and edge attack. Most early works focus on node attack, which often rank the nodes based on structural information such as collective influence (Morone & Makse, 2015), degree (Pastor-Satorras & Vespignani, 2001) and betweenness (Freeman, 1977). Unfortunately, the computational time of these methods increases exponentially as the network size expands. Furthermore, most heuristic methods (Braunstein et al., 2016; Hagen & Kahng, 1992; Luxburg, 2007; Mugisha & Zhou, 2016; Ren, Gleinig, Helbing, & Antulov-Fantulin, 2019; Zdeborová, Zhang, & Zhou, 2016b) are designed based on the intricate structure of ND tasks, and their generalization ability is often criticized for being difficult to meet the requirements of scenarios beyond their design purpose. To address these problems, researchers have incorporated node embeddings or edge embeddings into the ND process to accelerate it. Tan, Wu, Linyuan, Li, and Lu (2016)

* Corresponding author.

E-mail addresses: guofengwei@nudt.edu.cn (F. Guo), chenchao@nudt.edu.cn (C. Chen), zengli@nudt.edu.cn (L. Zeng), putianle@nudt.edu.cn (T. Pu), qf@nudt.edu.cn (F. Qing), fanchangjun09@163.com (C. Fan).

<https://doi.org/10.1016/j.eswa.2025.128121>

Received 28 December 2024; Received in revised form 31 March 2025; Accepted 8 May 2025

Available online 22 May 2025

0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

investigated network dismantling under conditions of incomplete link information and discovered that the integration of link prediction substantially improves the efficiency of the ND process. In recent years, the outstanding success of machine learning methods in the fields of computer vision and natural language processing has led researchers to turn their attention to using machine learning methods to solve the ND tasks. In particular, FINDER (Fan, Zeng, Sun, & Liu, 2020c) proposed by Fan et al. is a pioneering work in using deep reinforcement learning to solve ND tasks. They ingeniously modeled the ND tasks as a Markov Decision Process (MDP) and used graph neural networks (GNN) for representation. By learning from a vast amount of data, they revealed the efficient disruption strategies implied in the task.

Compared to node attack, little work has been proposed on edge attack (Fan et al., 2020a,b; Feng, Li, Zeng, & Liu, 2020a; Feng, Ren, Zeng, Yang, & Liu, 2020b; Feng, Zeng, Li, & Liu, 2020c; J. Zhang, & Han, 2017; Li, 2015; Wang, Feng, Zeng, & Liu, 2021; Xu, 2020). However, in real-world dismantling scenarios, edge attack is more prevalent, as nodes are typically fewer in number and better protected, making the cost of attacking them much higher than that of edges. Similarly, edge attack methods that use structural information, such as edge weights, betweenness, and so on, have been proposed successively (Cheng, Ren, Shen, Zhang, & Zhou, 2010; Girvan & Newman, 2002; Holme & Kim, 2002; Liu, Tang, Zhou, & Do, 2015; Onnela et al., 2007; Zhao et al., 2020). Among them, Qian, Li, Zhang, Ma, and Lu (2017) categorized edge attack methods based on structural information into seven types and pointed out that the importance of an edge can often be measured by the importance of nodes and the network. Chen, Wang, Zeng, Liu, and Chen (2023) used the Hadamard product of node pairs to represent edge embeddings in deep reinforcement learning. Based on the FINDER, an effective and flexible high-value edge attack sequence framework called SHEAR was created. However, it has not been tested on large networks and requires improved generalization.

Besides, Wang, Yang, Huang, and Xiao (2023) suggested converting the original graph into a line graph, and learning its node embeddings to obtain original edge embeddings. Inspired by the line graph transformation, we draw on high-performance node attack methods and design an end-to-end deep reinforcement learning framework named FIGHTER, which transforms edge attacks on the original graph to corresponding node attacks on the line graph. Given the NP-hard nature of the task, we treat the ND process as a MDP and learn the inherent efficient strategies through extensive data-driven learning. With the help of the line graph transformation, it is easy to identify the key edges in the original network. In addition, one of the most essential characteristics for evaluating networks is their robustness, which means that different network structures result in different levels of robustness (Artime et al., 2024b). The giant connected component (GCC) is used as a key indicator of the network's 'health' (Grassia & Mangioni, 2024). Unlike prior works, we not only consider the connected subgraph with the most nodes, but also the connected subgraph with the most edges, and the latter may be more meaningful in some scenarios. Take transport networks, for example, where one will spend more time on the edges than at the nodes. Here, we define the connected component with the most nodes as GCC_{node} and the connected component with the most edges as GCC_{edge} .

We also take the attack cost into account. Previous work mostly assumes that the cost of each edge is the same, but in some cases, the importance of edges in a network is different (Albert, Jeong, & Albert-Laszlo, 2000). Recently, a similar definition of cost was applied to study the fragmentation and strengthening processes in certain network models (Bröhl & Lehnertz, 2022). They discovered that the optimal cost for these processes is determined by the priority list of node degrees to be removed, which is independent of the network's degree distribution. Following this success, we define the cost of removing an edge as its degree in the line graph, which is numerically equal to the number of neighbor edges. In this paper, we conduct edge attacks on both synthetic and real-world networks, considering scenarios with both uniform and heterogeneous costs. The experimental results show that FIGHTER exhibits

superior performance and strong generalizability, making it quickly and widely applicable to the ND tasks.

Our main contributions are as follows:

- We introduce a novel GNN model that incorporates both edge and global information to produce high-quality edge feature vectors, which apply the node learning method GraphSAGE to the line graph and generate high-quality edge feature vectors directly from the edges themselves;
- We develop an efficient and flexible framework for searching high-value edge attack sequences, which can adjust reward function for different ND targets (attacking the connected subgraph with most nodes or edges) to suit various types and scales of ND tasks;
- We conduct extensive experiments to demonstrate the advantages of our method, surpassing the other five methods.

The structure of this paper as follows. Section 2 introduces related work. In Section 3, the ND problem and FIGHTER encoder, decoder, training and application modules are described. In Section 4, we compare the performance of six edge attack strategies including FIGHTER on six synthetic networks and five real networks, and analyze the time complexity. Finally, Section 5 gives conclusions.

2. Related works

For the network dismantling (ND) works of edge attack, we divide them into several categories:

2.1. Structural information-based methods

- Early works focused on traditional algorithms identifying key edges based on structural information. The common four edge attack strategies include random adaptive (RA) (Albert et al., 2000), high degree sum adaptive (HDSA) (Holme & Kim, 2002), high degree product adaptive (HDP) (Holme & Kim, 2002) and high edge betweenness (HEB) (Girvan and Newman (2002)). RA removes all edges in a random order. HDSA or HDP defines the edge's importance by the sum or product of its endpoints' degrees. HEB quantifies edge importance by the number of shortest paths passing through it and provides a precise global measure. In addition, some researchers have innovated indicators for measuring centrality (Bröhl & Lehnertz, 2022; Kanwar, Kaushal, Kumar, Gupta, & Khari, 2022; Pournajar, Zaiser, & Moretti, 2022). They considered the topological structure of edges to determine the ranking of edge importance (Singh, Nandi, Chakraborty, & Choudhury, 2022; Yu, Chen, & Zhao, 2018; Zhou, Lv, Wang, Zhang, & Xuan, 2022). Although these algorithms are easy to operate and have strong applicability, they have some problems, such as unsatisfactory results in most cases.

2.2. Heuristic-based methods

- Recently, researchers have introduced heuristic algorithms to address the poor performance of traditional methods, aiming to find near-optimal edge attack strategy. The Hierarchical Power Iterative Normalized cut (HPI-Ncut) algorithm was proposed by Ren et al, and degree cost segmentation effectiveness (DCFE) was used to measure its performance (Ren, Gleinig, Tolić, & Antulov-Fantulin, 2018). Peng et al. introduced point-line transformation, designed edge collective influence (ECI), improved edge collective influence (IECI) and improved edge collective influence with reinsertion (IECIR) for tasks such as hiding and fast dismantling targets (Peng, Fan, Ren, & Linyuan, 2023). Furthermore, several works investigated the impact of repair operations on network robustness after being attacked (North, 2021; Topîrceanu, Udrescu, & Mărculescu, 2020; Zhao, Sun, Shen, & Xia, 2021). Although network dismantling methods based on heuristic algorithms are more effective than those only based on structural information, heuristic algorithms face issues like poor applicability and low solving efficiency.

2.3. Learning-based methods

• Finding the optimal edge attack sequence in the network can also be regarded as a combinatorial optimization problem on the graph, which is an essential topic in machine learning and operation research. Finding the high-value edge attack sequence by combining graph neural network (GNN) and deep reinforcement learning (DRL) is also a prevailing research area. Chen et al. used the Hard-mard product of nodes at both ends of the edge to represent the edge embeddings and designed an edge attack framework called SHEAR based on the FINDER framework (Chen et al., 2023). Based on the FINDER framework, several targeted network dismantling algorithms have been developed: Zeng et al. enhanced SHATTER framework for heterogeneous networks (Zeng, Liu, Lu, Chen, & Zhou, 2021). The MINER framework was created by Zeng et al. for multiplex networks (Zeng, Lu, Liu, Chen, & Zhou, 2022). Mao et al. focused on identifying vulnerable nodes in urban multilayer networks (Mao et al., 2023). The HITTER is developed by Yan et al. for hypergraph networks (Dengcheng, Wenxin, Yiwu, Qiang, & Yun, 2022). A new edge removal framework called eGDM-RL was developed by Marco et al, which combines geometric deep learning and reinforcement learning (Grassia & Mangioni, 2024). The method employs Graph Attention Networks (GAT) and Deep Q-Learning (DQN) to identify the edges whose removal maximizes the degradation of network performance, thereby optimizing the edge attack sequence. However, they did not conduct experiments on the large graph, and the flexibility and generalization of their frameworks need to be improved.

Above all, deep reinforcement learning offers high efficiency and strong generalization, making it ideal for network dismantling (ND) research in complex networks. Nonetheless, the existing edge attack frameworks need enhancement and the flexibility and generalization also need to be improved. Therefore, we design the edge attack framework FIGHTER, which can be flexibly adapted to various ND tasks and applied to large networks.

3. Model

In this section, FIGHTER is developed to search high-value edge attack sequence based on DQN, which is shown in Fig. 1.

3.1. Problem formalization

Two types of network dismantling (ND) tasks are formulated: attack the connected subgraph with most nodes or most edges. Fig. 2 shows that the Giant Connected Component (GCC) in the original graph has the most nodes, while the line graph has the most edges. The size of GCC_{node} and GCC_{edge} are numerically equal to the original graph GCC size and the line graph GCC size. Diverse ND objectives are formulated for the above two scenarios. By attacking the original graph or the line graph, we can quickly break down the subgraph with either the most nodes or the most edges.

For network $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ and $\mathcal{E}$ are the set of points and the set of edges respectively, A is the adjacency matrix of the edges, $X^{|\mathcal{E}| \times c}$ is the edge attribute matrix and c is the dimension of $\mathcal{E}$ the attribute matrix. The ND process can be seen as a process of removing a sequence of edges $\{e_1, e_2, \dots, e_k\}$ in the network. Our framework can deal with any well-defined connectivity measure $\mu : \{G\} \rightarrow \mathbb{R}$, which maps a graph into a non-negative real number. We consider two measures: (i) the size of the original graph GCC (GCC_{node}): $\mu(G) = \max\{\delta_i; C_i \in G\}$; (ii) the size of the line graph GCC (GCC_{edge}): $\mu(G) = \max\{\delta_i; C_i \in L(G)\}$. The C_i is the i th connected component in current graph and δ_i is the size of C_i , corresponding to the network dismantling (ND) problem. The

termination condition for FIGHTER is that all edges in the network are removed.

In brief, our task is acquiring an edge attack sequence which can minimize the value of accumulated normalized connectivity (ANC). Formula (1) represents the function. When considering the attack cost, formula (2) is used. The edge attack cost is defined as its degree in the graph, which is numerically equal to the number of its neighbor edges.

Our task is to obtain an edge attack sequence that minimizes the accumulated normalized connectivity (ANC) value of the original graph G or the line graph $L(G)$:

$$R(e_1, e_2, \dots, e_E) = \frac{1}{|\mathcal{E}|} \sum_{k=1}^{|\mathcal{E}|} \frac{\mu(G \setminus \{e_1, e_2, \dots, e_k\})}{\mu(G)} \quad (1)$$

where, $\mu(G)$ is the initial GCC size of original graph G or line graph $L(G)$ and $\mu(G \setminus \{e_1, e_2, \dots, e_k\})$ is the GCC size of original graph G or line graph $L(G)$ after removing the edges according to the set $\varphi\{e_1, e_2, \dots, e_k\}$. The value of R can be regarded as the area under the ANC curve, which is plotted with the horizontal axis being $k/|\mathcal{E}|$ and the vertical axis being $\mu(G \setminus \{e_1, e_2, \dots, e_k\})$.

In some application scenarios, attacking different edges have different costs, we can use the degree of the edge in the line graph as the attack cost:

$$R_{cost}(e_1, \dots, e_E) = \sum_{k=1}^{|\mathcal{E}|} \frac{\mu(G \setminus \{e_1, \dots, e_k\})}{\mu(G)} c(e_k) \quad (2)$$

where, $c(e_k)$ is the normalized removal cost associated with edge e_k and $\sum_{k=1}^{|\mathcal{E}|} c(e_k) = 1$. Note that R is a special case of R_{cost} , where $c(e_k) = 1/|\mathcal{E}|$. The range of both R and R_{cost} are between 0 and 1.

The R value represents the result of the algorithm for the network dismantling (ND) tasks and the change of R value mirrors the trend of ANC curve. Therefore, the algorithm performs better with smaller values of R and R_{cost} for the same network.

3.2. Architecture: FIGHTER

In this section, we develop FIGHTER to search high-value edge attack sequence based on Deep Q-Learning (DQN). FIGHTER consists of five modules: Line graph transformation, Encoder, Decoder, Offline Training and Real-world Application. The details are shown as follows.

In the line graph transformation module, the original graph is converted into a line graph through a node-edge transformation process. This line graph more effectively captures the connectivity between edges in the original graph, treating these edges as nodes, which enables the GraphSAGE aggregation within the encoder module.

To represent the states and actions, we design the encoder module. The GraphSAGE is employed to embed the nodes in the line graph into a low-dimensional space, effectively achieving original graph edge embedding. To capture more graph information, a virtual node is introduced that considers all real nodes as neighbors in the line graph to represent the entire line graph.

For the decoder module, Q function is designed, which utilizes the encoder's embeddings of states and actions to calculate a score that evaluates the actions. Specifically, the outer product operation is applied on the embeddings of states and actions to model finer state-action dependencies. Subsequently, a multi-layer perceptron (MLP) with rectified linear unit (ReLU) activation is used to map the outer product to a scalar value to calculate the score of the action.

FIGHTER is offline trained on small Barabási-Albert (BA) synthetic graphs. For each graph, the FIGHTER considers finding key edges as a Markov Decision Process (MDP): interacting with the environment through a sequence of states, actions and rewards. An MDP can be represented as a quintuple (S, A, R, P, γ) , where S denotes the set of possible states, A represents the set of possible actions, R signifies the distribution of rewards given a state and action pair, P indicates the transition probability distribution over the next state given the state and action

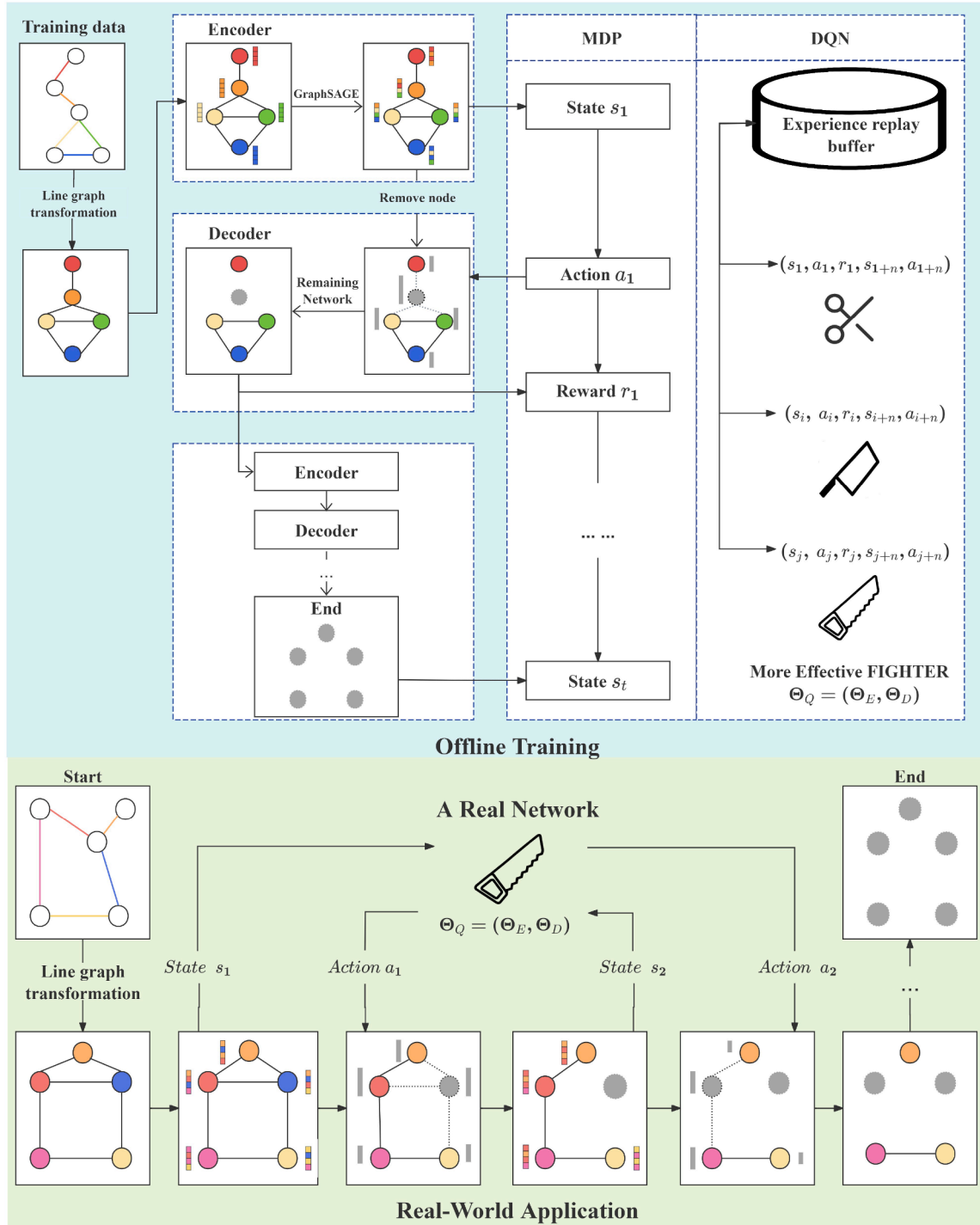


Fig. 1. The framework of FIGHTER. The whole framework consists of two phases: Phase-1: Offline training, during which the agent FIGHTER is trained to perform well on BA synthetic graphs; Phase-2: Real-world application, during which the well-trained agent is applied to a real-world network to find the key edges.

pairs and $\gamma \in (0, 1)$ is a discount factor that determines the importance of future rewards. For the FIGHTER, the environment is the network being analyzed, the state is defined as the residual network, the action is to remove the edge, and the reward is the decrease of the is the reduction ratio of GCC_{node} size or GCC_{edge} size after taking the action. During the process, the FIGHTER constantly updates its parameters and becomes more intelligent in solving the task. All parameters can be adjusted by training small synthetic graphs offline and modifying their online execution.

When applied to a real-world network, FIGHTER will simply repeat a greedy procedure to return the optimal edge attack sequence.

3.2.1. Line graph transformation

First of all, we transform the training networks into line graphs. The line graph reveals the connection relationship between edges and facilitates us to treat them as nodes for GraphSAGE aggregation processing later.

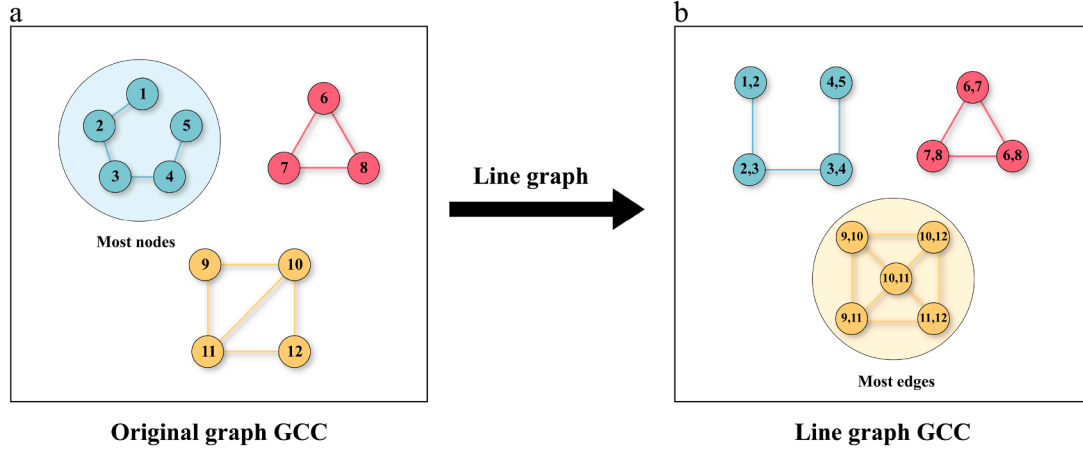


Fig. 2. The GCC of the original graph and the line graph. Figure a is the original graph, while Figure b is the line graph derived from Figure a through node-to-edge transformation. The original graph GCC has the most nodes while the line graph GCC has the most edges, the size of GCC_{node} and GCC_{edge} are numerically equal to the original graph GCC size and the line graph GCC size.

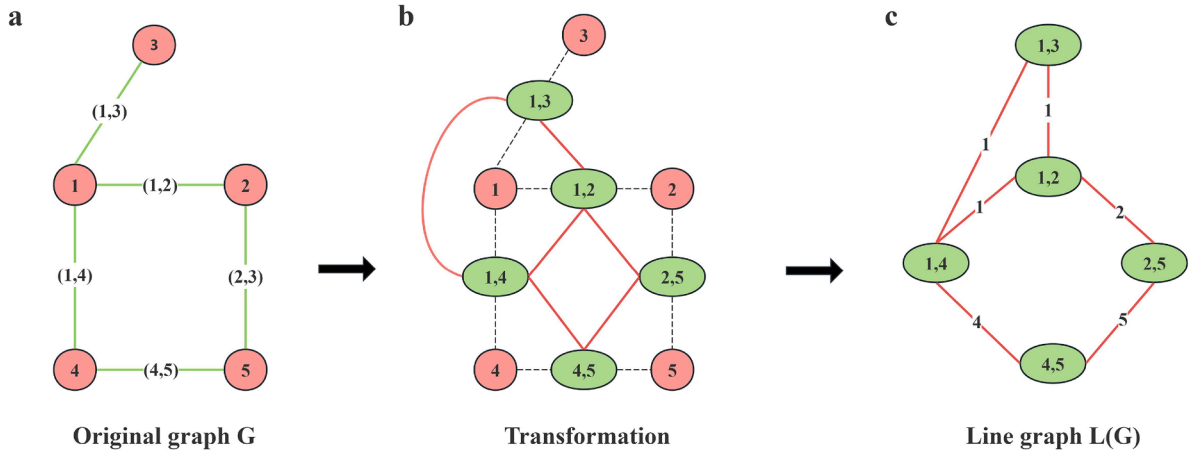


Fig. 3. Line graph transformation. Figure a shows the original graph. Figure b illustrates the node-to-edge transformation process: each edge in the original graph becomes a node in the line graph. If two edges are adjacent to each other in the original graph, connect an edge between the corresponding nodes in the line graph. Figure c presents the final line graph after the node-to-edge transformation.

For a graph $G = (\mathcal{V}, \mathcal{E})$, the line graph $L(G)$ shows how the edges are connected. Each edge in G becomes a node in $L(G)$, where two nodes are connected if their corresponding edges in G share an endpoint. The specific process is shown in Fig. 3. The line graph retains all edges and connections of the original graph but may lose isolated nodes and nodes with degree 1 from the original graph, as seen with node 3 in Fig. 3. However, these nodes are not critical to the graph structure. Thus, the loss during line graph transformation is acceptable and does not affect result accuracy.

Wang et al. (2023) demonstrated that the original graph could be converted into a line graph, and the edge embeddings of the original graph could be derived by learning the node embeddings of the line graph. Furthermore, line graph more directly shows the connected subgraph with the most edges (GCC_{edge}), because the size of the connected subgraph with the most edges (GCC_{edge}) is numerically consistent with line graph GCC size.

3.2.2. Encoder

In our experiments, GraphSAGE is used to obtain node embeddings in the line graph, which is the edge embedding of the original graph. In the encoder module, a GraphSAGE method is designed, where $\Theta_E = (W_1, W_2, W_3)$ is the parameter in the module. The detail is shown in Algorithm 1.

We transform the BA networks used for training into line graphs to obtain the node embeddings of the line graph, which are the edge embeddings of the original graph.

The node embedding vector of line graph is shown as follow:

$$b_e^{(0)} \leftarrow \text{ReLU}(X_e \cdot W_1), \quad \forall e \in \mathcal{E} \quad (3)$$

Regularized by formula as follow:

$$b_e^{(0)} \leftarrow b_e^{(0)} / \|b_e^{(0)}\|_2, \quad \forall e \in \mathcal{E} \quad (4)$$

Through the line graph, we can use the GraphSAGE embedding method to aggregate the k-hop neighbor information of each edge in the original graph as follow:

$$\begin{aligned} b_{\mathcal{N}(e)}^{(k'-1)} &\leftarrow \text{AGG}_{j \in \mathcal{N}(e)} b_j^{(k'-1)} \\ b_e^{k'} &\leftarrow \text{ReLU}([W_2 \cdot b_e^{(k'-1)}, W_3 \cdot b_{\mathcal{N}(e)}^{(k'-1)}]) \end{aligned} \quad (5)$$

In the formula above, the k' is the iteration times, $\mathcal{N}(e)$ is the neighbor definition function, AGG is the aggregate function and we set $\text{AGG} \leftarrow \text{sum}$.

When the number of iterations reaches k, regularize $b_e^{(k)}$ as follow:

$$b_e^{(k)} \leftarrow b_e^{(k)} / \|b_e^{(k)}\|_2, \quad \forall e \in \mathcal{E} \quad (6)$$

The $\forall e_y \in \mathcal{E}$ is the set of final embedding vectors that aggregate the edges of the structure information and attribute information of the network.

In order to get the embedded vector representation of the network $\mathcal{G}$, the method of embedding is designed as follow:

$$b_s = \text{AGG}(b_e^k, \forall e \in \mathcal{E}) \quad (7)$$

We set the AGG function to the sum function and introduce the virtual node h_s that aggregate all the edge-embeddings (node embeddings for the line graph), representing the embeddings for the entire network.

Algorithm 1 Encoder.

```

1: Input: Edge features  $X_e$  and aggregate function  $\text{AGG}^{(k)}$ .
2: Output: Embedding vector  $b_e, \forall e \in \mathcal{E} \cup \{s\}$ .
3: Initialize learnable parameters  $W$ .
4:  $\mathbf{b}_e^{(0)} \leftarrow \text{ReLU}(X_e \cdot \mathbf{W}_1), \mathbf{b}_e^{(0)} \leftarrow \mathbf{b}_e^{(0)} / \|\mathbf{b}_e^{(0)}\|_2, \forall e \in \mathcal{E} \cup \{s\}$ 
5: for  $k_i = 1$  to  $k$  do
6:   for  $e \in \mathcal{E}$  do
7:      $\mathbf{b}_{N(e)}^{k'} \leftarrow \text{AGG}_{k'}(\{\mathbf{b}_e^{(k'-1)}, \forall e \in \mathcal{N}(e)\})$ 
8:      $\mathbf{b}_e^{(k')} \leftarrow \text{ReLU}(\mathbf{W}_2 \cdot \mathbf{b}_e^{(k'-1)}, \mathbf{W}_3 \cdot \mathbf{b}_{N(e)}^{(k'-1)})$ 
9:   end for
10: end for
11:  $b_e \leftarrow b_e^k / \|b_e^k\|_2, \forall e \in \mathcal{E} \cup \{s\}$ 

```

3.2.3. Decoder

In the decoder module, the process of network dismantling is regarded as a Markov decision process and the state of the network at step t is s_t .

The action $a_t \leftarrow \text{delete } e_t$ is to delete the edge in the original graph. After the edge removed, the network is encoded again to obtain the new state s_{t+1} . The reward of $Q(s, a)$ is the Q value of the action mapped to the true value by the Q function to satisfy the downstream task.

The cumulative return of actions performed at state s_t as follow:

$$Q(s_t, a_t) = W_5^T \text{ReLU}(b_y^T h_s W_4)$$

b_y and h_s are the action and state vectors output by the encoder module, $\text{ReLU}(z) = z$ if $z > 0$ and 0 otherwise, $\Theta_D = (W_4, W_5)$ are trainable decoder parameters.

3.2.4. Offline training

FIGHTER is trained over 20,000 Barabási-Albert (BA) small synthetic graphs of 30–50 nodes. In order to perform the end-to-end learning of the parameters in the encoder and decoder, we combine the n-step Q-learning loss in formula (10) and the graph reconstruction loss in formula (12) to form the loss function in formula (13). The n-step Q-learning loss minimizes the gap between Predicted Q values (Q_{pred}) and Target Q values (Q_{target}), and the graph reconstruction loss preserves the line graph structure. We create four agents to handle two connectivity measures: original graph GCC size (GCC_{node}) and line graph GCC size (GCC_{edge}), under two scenarios: edge-unweighted and edge-line graph degree-weighted. All agents share the same training procedure except for the reward function, which is determined by the respective ANC. Formulas (8) and (9) describe the reward function, and Algorithm 2 details the whole training procedure.

(1) Reward r

For the two different cases in Section 3.1, FIGHTER sets two different reward functions as follow. Different reward functions are designed for different network disintegration tasks. The choice of reward function is determined at training initiation. The formula (8) was used in the uniform-removal-cost attack scenario, while the formula (9) was used in the heterogeneous-removal-cost attack scenario.

$$r = \frac{1}{|\mathcal{E}|} \cdot \frac{\mu(\mathcal{G}'/e)}{\mu(\mathcal{G})} \quad (8)$$

Algorithm 2 Offline Training.

```

1: Initialize experience replay buffer  $D$  with size  $L$ 
2: Initialize action-value function  $Q$  with parameters  $\Theta_Q = (\Theta_E, \Theta_D)$ ,  $\Theta_E = (W_1, W_2, W_3)$  and  $\Theta_D = (W_4, W_5)$ 
3: Initialize target action-value function  $\hat{Q}$  with parameters  $\Theta_Q$ 
4: for episode = 1 to  $N$  do
5:   Generate a random graph with the BA model and convert it into line graph.
6:   Preprocess the state representation  $s$  of it
7:   Initialize states to an empty sequence  $S = ()$ 
8:   for  $t = 1$  to  $T$  do
9:     With probability  $\epsilon$  select a random action  $a_t$ 
10:    otherwise select  $a_t = \arg \max_a Q(s_t, a; \Theta_Q)$ 
11:    Execute action  $a_t$  in emulator, observe reward  $r_t$ 
12:    Get the remaining network  $\mathcal{G}_{t+1} = \mathcal{G}_t \setminus \{e_t\}$  and its state representation  $s_{t+1}$ 
13:    Store transition  $(s_t, a_t, r_t, s_{t+1})$  in  $D$ 
14:    Sample a random transition  $(s_j, a_j, r_j, s_{j+1})$  from  $D$ 
15:    Set  $y_j = \begin{cases} r_j & \text{if terminates at } s_{j+1} \\ r_j + \gamma \max_{a'} Q(s_{j+1}, a'; \hat{\Theta}_Q) & \text{otherwise} \end{cases}$ 
16:    Perform stochastic gradient descent on formula (14) with respect to the parameters  $\Theta_Q$ 
17:    Each  $C$  steps reset  $\hat{Q} = Q$ 
18:   end for
19: end for

```

$$r_{\text{cost}} = \frac{\mu(\mathcal{G}'/e)}{\mu(\mathcal{G})} \cdot c(e) \quad (9)$$

where, $|\mathcal{E}|$ is the number of edges, $\mu(\mathcal{G})$ is the GCC size of the initial original graph $\mathcal{G}$ or line graph $L(\mathcal{G})$, $\mu(\mathcal{G}'/e)$ is the GCC size of the current original graph $\mathcal{G}'$ or line graph $L(\mathcal{G}')$ after removing edge e and $c(e)$ is the cost of attacking edge e .

(2) Q learning loss

In the process of reinforcement learning, the parameter of the target network is $\Theta_Q = \{\Theta_E, \Theta_D\}$. The Batch Gradient Descent (BGD) method is used to update the learning parameter.

The Q-learning loss function is designed as follows:

$$Q = r_{t,t+n} + \gamma \max_{a'} \hat{Q}(s_{t+n}, a'; \hat{\Theta}_Q) - Q(s_t, a_t; \Theta_Q) \quad (10)$$

The target network parameter is $\hat{\Theta}_Q$. The batch training sample of Q learning loss is randomly selected from the experience replay pool $(s_t, a_t, r_{t,t+n}, s_{t+n}) \sim D(L)$, the parameters of Q network Θ_Q are set every n step by the target network and remain unchanged. The discount factor γ is used to determine the future reward weight.

(3) Graph reconstruction loss

The function of the graph reconstruction loss is used to maintain the structure of the line graph in the embedding space, thereby preserving the connections between edges in the original graph.

$$G_{ij} = \|b_i - b_j\|_2 \quad (11)$$

$$G_{\text{loss}} = \sum_{i,j}^{|\mathcal{E}|} a_{ij} G_{ij}^2 \quad (12)$$

where, G_{ij} represents the reconstruction loss of the graph, a_{ij} represents whether the two edges e_i and e_j in the line graph are connected, y_i and y_j is their representation vector.

(4) Loss function

The loss function of the whole is defined as the sum of the loss function of reinforcement learning and the loss function of graph reconstruction:

$$\text{Loss} = E_{(s_t, a_t, r_{t,t+n}, s_{t+n}) \sim D(L)} (Q^2) + \alpha \sum_{i,j}^{|\mathcal{E}|} a_{ij} G_{ij}^2 \quad (13)$$

Loss function contains two parts: (i) Q learning loss is used to minimize the gap between Predicted Q values (Q_{pred}) and Target Q values (Q_{target}); (ii) Graph reconstruction loss is used to maintain the structure information of the graph. α is a positive hyper-parameter that weighs between these Q learning loss and graph reconstruction loss.

3.2.5. Real-world application

According to the different reward functions r , a set of model parameters Θ_Q with the best parameter effect are saved after training, and then the model is directly applied to the synthetic graphs and the real networks to get the edge attack strategy. For the test network G , the R output of the model can be directly used as the basis for judging the result.

During the training period, the BA graphs are employed for training and validation.

We adopt an ϵ -greedy policy with ϵ linearly annealing from 1.0 to 0.05 over 10,000 episodes to balance exploration and exploitation. In the application phase, a batch edges greedy policy is utilized, which removes the top 1% highest Q edges at each adaptive step until the game concludes.

The model is trained for a total of one hundred thousand episodes and use an experience replay buffer of size 50,000 to store the most recent transitions. Every 300 episodes, the agent is evaluated on 200 synthetic graphs of the same scale as the training graphs, and the performances are averaged.

After each episode, we randomly sample a mini-batch of 4-tuple transitions from the experience replay buffer and perform the stochastic gradient descent update to minimize the loss function. Algorithm 3 details the whole procedure.

Algorithm 3 Real-world application.

```

1: Input: The network to be disintegrated  $G(\mathcal{V}, \mathcal{E})$ .
2: Output: The high-value edge attack sequence  $\eta$ .
3: Convert  $G$  into line graph  $L(G)$ .
4: Load the trained parameters  $\Theta_Q = \{\Theta_D, \Theta_E\}$  to the function  $Q$ 
5: Set  $e = 0$ 
6: for epoch = 1 to 100 do
7:   Select the edges need to be attacked
8:   Calculate  $R$  Value
9: end for
10: Get the high-value edge attack sequence  $\eta$  of network  $G(\mathcal{V}, \mathcal{E})$ 

```

4. Experiments

In this section, the trained FIGHTER is applied to a number of different networks to verify its superiority. At the same time, five comparison methods are used to compare the same scene. All experiments are run on a computer equipped with GPU NVIDIA GeForce RTX 4060. Training the model on the server took about 72 hours. More than 20,000 graphs are trained for the model, and the setting of hyperparameters is shown in Table 1.

4.1. Datasets

We evaluated FIGHTER on 23 datasets, which can be divided into two types: synthetic graphs and real networks.

To validate the scalability of our algorithm to large-scale networks, we conduct experiments on synthetic datasets comprising 6 clusters,

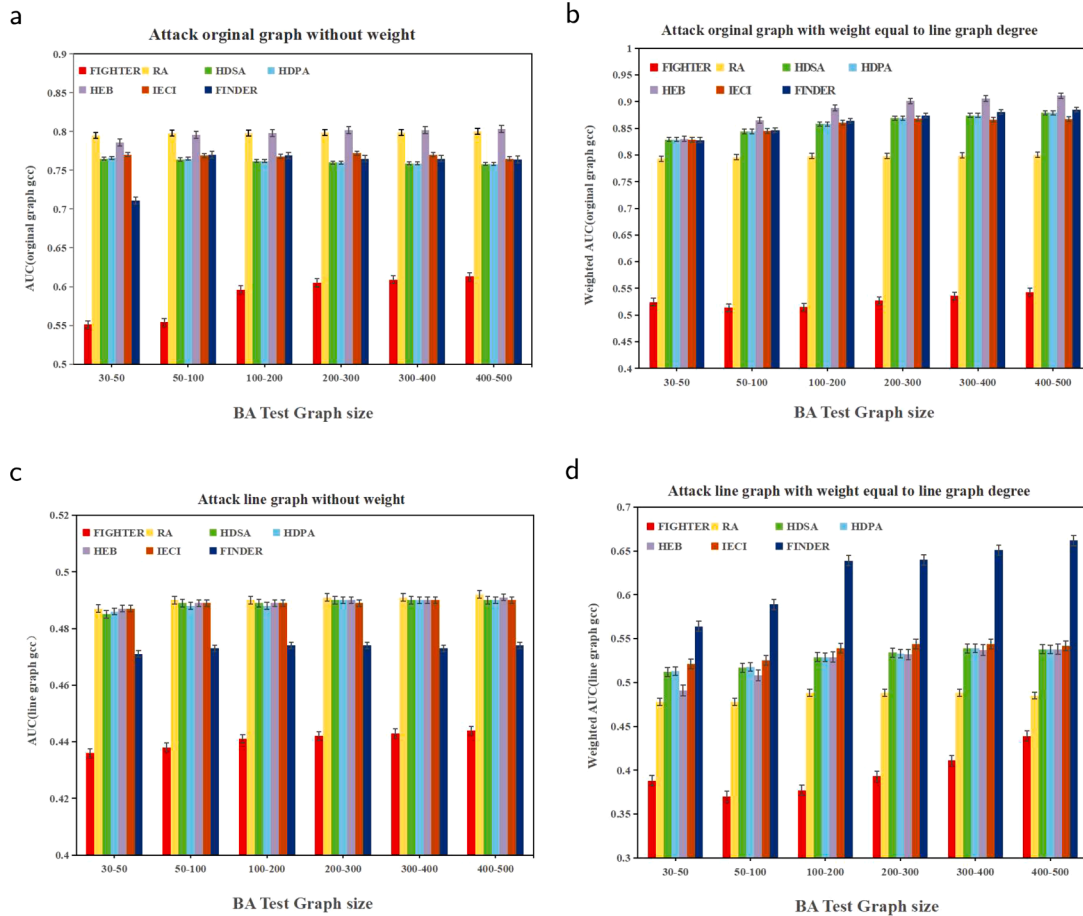


Fig. 4. Performance of FIGHTER on BA synthetic graphs. In all the cases, FIGHTER was trained on BA graphs of 30–50 nodes. For each scale, we randomly generate 100 instances, and report the average results over them.

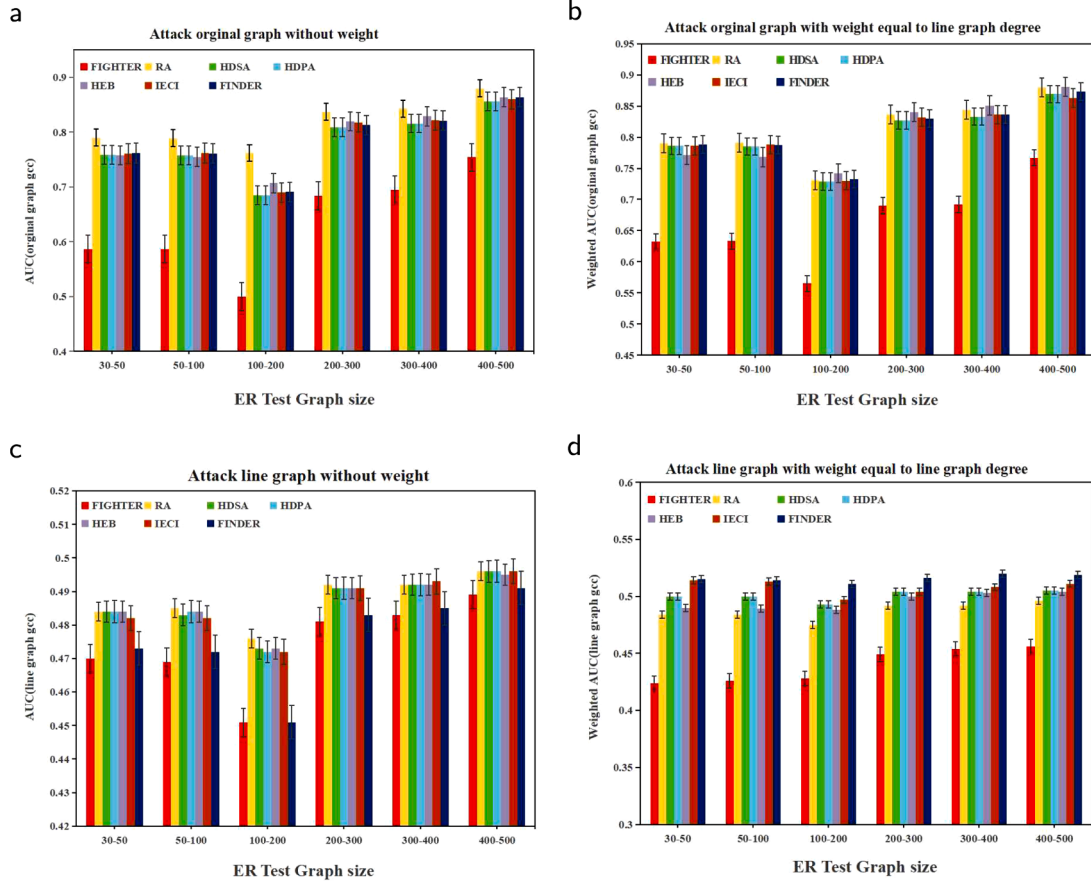


Fig. 5. Performance of FIGHTER on ER synthetic graphs. In all the cases, FIGHTER was trained on ER graphs of 30–50 nodes. For each scale, we randomly generate 100 instances, and report the average results over them.

Table 1

Hyperparameters setting of FIGHTER. All parameters are obtained by an informal search. Due to the high computational cost, we didn't perform a systematic grid search for the optimal set of hyper-parameters.

Parameter	Value	Description
Learning rate	0.0001	Learning rate adopted by optimizers.
Alpha	0.001	Weight of reconstruction loss.
Batch size	64	Number of samples in batch training.
Max iteration	100,000	Iteration times of DQN training.
Gamma	1	Decay rate of past observations.

each containing 100 Barabási-Albert (BA) networks generated randomly. These networks are categorized based on size into the following intervals: 30–50 nodes, 50–100 nodes, 100–200 nodes, 200–300 nodes, 300–400 nodes, and 400–500 nodes. Furthermore, the same experiments are also conducted on Erdős-Rényi (ER) synthetic networks and Watts-Strogatz (WS) synthetic networks.

The real datasets include Krebs, Dolphins, Moreno, Univ and Digg, taking the edge order $10^2 \sim 10^6$ as the experimental scope.

- **Krebs** is a terrorist network (Arulselvan, Commander, Elefteriadou, & Pardalos, 2009), where members communicate through corresponding channels. Nodes represent terrorist members, while edges represent connections between them.
- **Dolphins** is a social relationship network of dolphins developed by Kunegis (2013). Through observing 62 dolphin groups in Doubtful Sound Strait, New Zealand for up to 7 years. The nodes represent dolphins and the edges represent frequent contact between them.

Table 2

Parameters of real network.

Dataset	Krebs	Dolphins	Moreno	Univ	Digg
N	62	62	829	1133	29,652
E	152	159	1476	5451	84,781
M	22	12	25	71	310
$\langle k \rangle$	4.90	1	3.55	9	5.72
C	0.4805	0.2590	0.0058	0.2202	0.0054

- **Moreno Crime** consists of criminal networks with 829 nodes and 1476 edges (Kunegis, 2013). The nodes represent suspects, while the edges represent their criminal relationships.
- **Univ** is a social network about a university's Email exchange which contains 1133 nodes and 5451 edges (Kunegis, 2013). Each node represents a person and each edge represents an Email relationship between two people.
- **Digg** is a reply network for the social news site, comprising 30,398 nodes and 87,627 edges (Kunegis, 2013). Each node represents a user of the site, while each edge denotes that a user replied to another user.

The specific parameters of these real networks are shown in Table 2. N is the number of the nodes, E is the number of the edges, M is the maximum degree, $\langle k \rangle$ is the average degree and C is the clustering coefficient.

4.2. Baseline

Five methods are chosen as the baseline, namely random adaptive (RA) (Albert et al., 2000), high degree sum adaptive (HDSA) (Holme

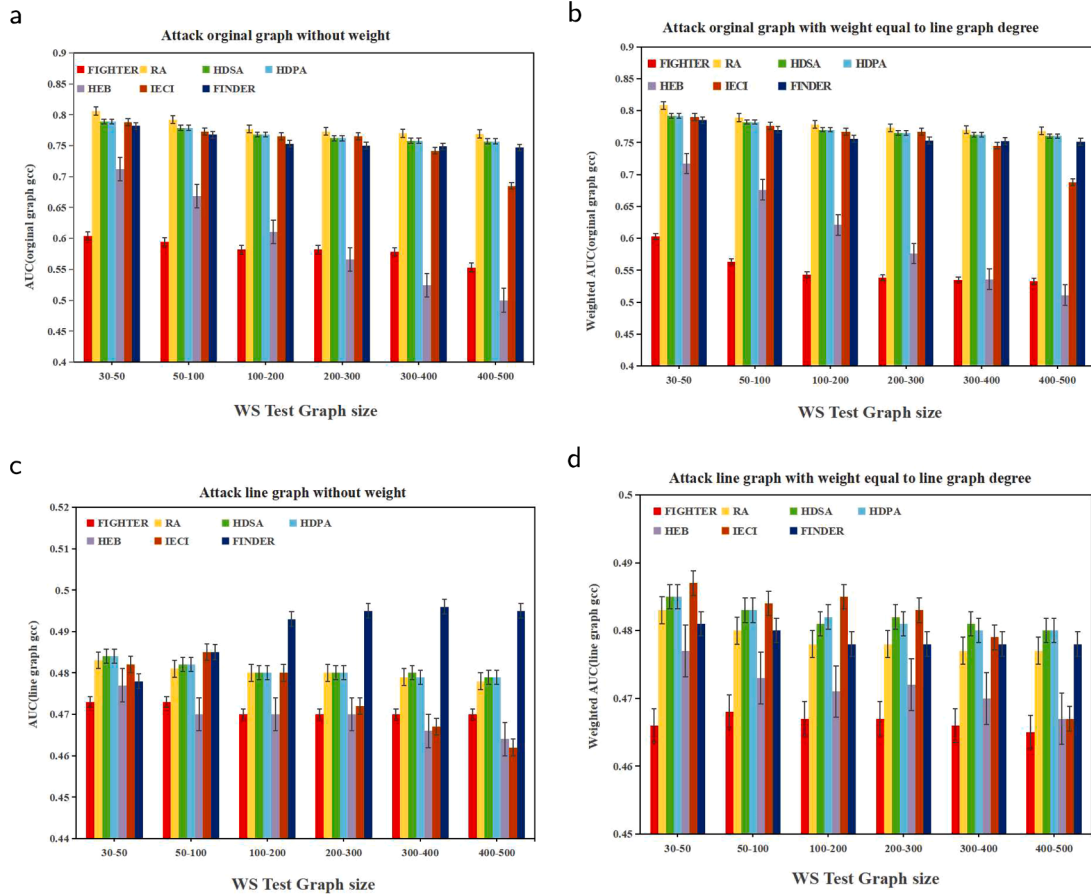


Fig. 6. Performance of FIGHTER on WS synthetic graphs. In all the cases, FIGHTER was trained on WS graphs of 30–50 nodes. For each scale, we randomly generate 100 instances, and report the average results over them.

& Kim, 2002), high degree product adaptive (HDPA) (Holme & Kim, 2002), high edge betweenness (HEB) (Girvan & Newman, 2002) and improved edge influence attack (IECI) (Peng et al., 2023). These methods have been introduced in Section 2.

- **Random Adaptive (RA)** randomly removes one edge at a time until all edges in the original graph are removed (Albert et al., 2000).
- **High Degree Sum Adaptive (HDSA)** ranks edges by the sum of their endpoints' degrees and removes the edge with the highest degree sum (Holme & Kim, 2002). HDSA recomputes the degree sum of the remaining edges after each edge removal, and the termination condition is that all edges are removed.
- **High Degree product Adaptive (HDPA)** is basically the same as the HDSA algorithm, except that search edge with the maximum degree sum are converted to search edge with the maximum degree product (Holme & Kim, 2002).
- **High Edge betweenness (HEB)** removes the edges according to their betweenness centrality (BC) (Girvan & Newman, 2002). Note that BC of an edge equals the sum of the fraction of all-pairs shortest paths that pass through this edge, and the Brandes algorithm is utilized to compute these values simultaneously (Brandes, 2008). Recalculating after each edge removal is computationally expensive for medium and large networks, so the betweenness are calculated once and edges are removed sequentially according to the betweenness value.
- **Improved Edge Collective Influence (IECI)** is a network dismantling method based on the collective behaviour of network edge (Peng et al., 2023). It transforms the original graph into a line graph derived from its adjacency list and evaluates the influence of all edges at the beginning. Subsequently, IECI sequentially removes the

edge with the highest CI value in the line graph and recalculates the CI values of the remaining edges until the original graph's GCC size drops below 1%. If the original graph's GCC size is less than 100, the IECI concludes after removing all edges. Otherwise, some edges are reinserted into smaller connected subgraphs to enhance the results after edge removal. Ultimately, the IECI algorithm employs a summation rule to reintegrate specific edges into smaller clusters post-dismantling.

- **FINDER** is a deep reinforcement learning algorithm for identifying critical nodes in complex networks (Fan et al., 2020c). It is offline trained on small synthetic graphs, framing critical node identification as a Markov decision process. It collects samples and updates parameters through state-action-reward interactions. During training, graph representation learning encodes network structure information, and a deep parameterized Q-function is designed for decoding. In application, it repeats a greedy process in real-world networks, selecting nodes in batches to efficiently identify critical nodes. We used FINDER to process the line graph and improving the FINDER algorithm for finding key edges.

4.3. Result

4.3.1. Uniform-removal-cost attack scenario

For the task of attacking the connected subgraph with most nodes (GCC_{node}), results of attacking synthetic graphs' original graph and real networks' original graph are recorded in Figs. 4(a) and 7. Compared with the best results of other algorithms, FIGHTER has an average improvement of 22.75% in the BA graphs and an average effect improvement of 17.09% in the real networks, which is significantly ahead of the other

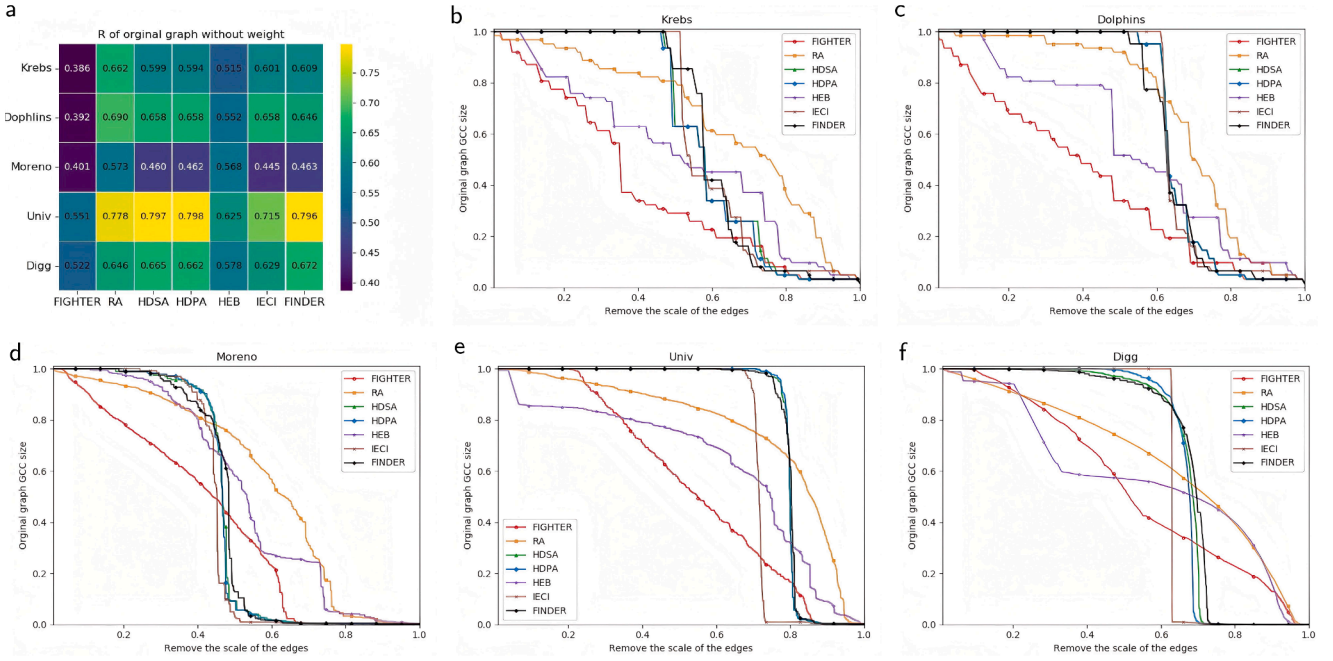


Fig. 7. The original graph's R value and decline curve under edge attack without weight. The abscissa is the proportion of the removed edges, and the ordinate is the ratio of GCC_{node} size after the edge are removed to the GCC_{node} size at the beginning.

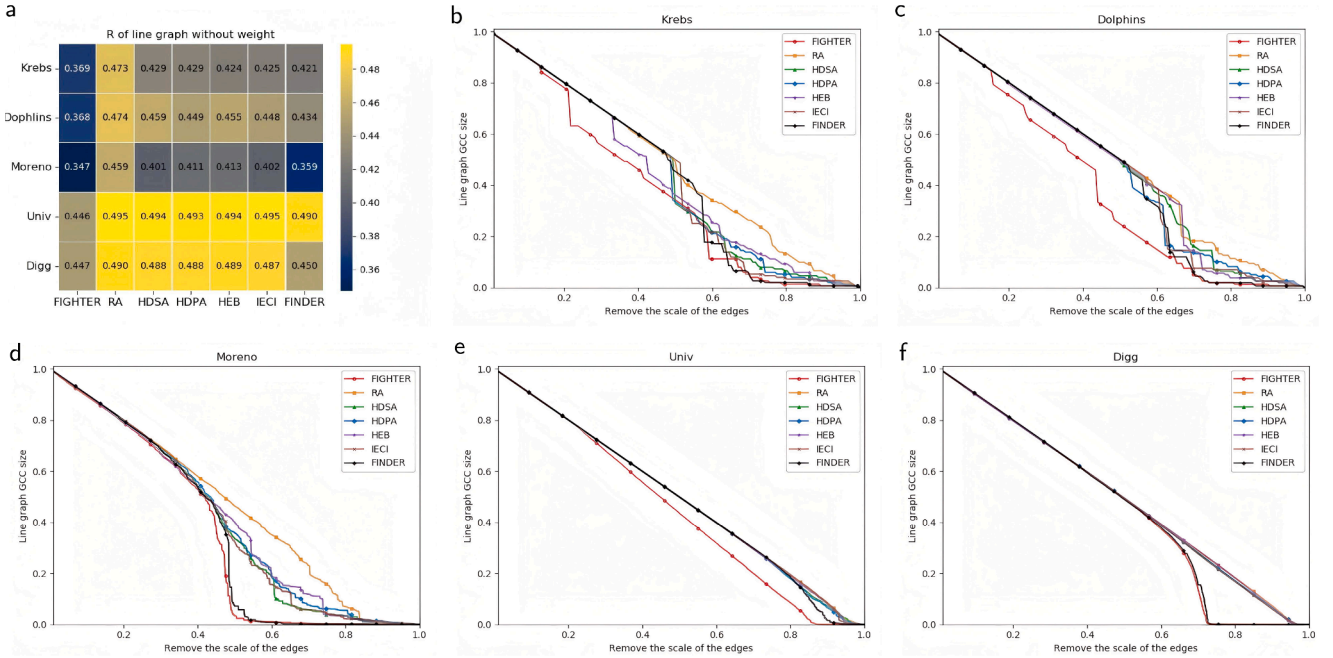


Fig. 8. The line graph's R value and decline curve under edge attack without weight. The abscissa is the proportion of the removed edges, and the ordinate is the ratio of GCC_{edge} size after the edges are removed to the GCC_{edge} size at the beginning.

algorithms. Besides, for the other types of synthetic networks, FIGHTER has an average improvement of 18.69 % in the ER graphs and an average effect improvement of 2.51 % in the WS graphs, which exhibits strong generalization capability. The details are recorded in Figure Figs. 5(a) and Figure Figs. 6(a).

Furthermore, for the task of attacking the connected subgraph with most edges (GCC_{edge}), results of attacking synthetic graphs' line graph and real networks' line graph are recorded in Figs. 4(c) and 8. Compared with the best results of other algorithms, FIGHTER has an average improvement of 6.77 % in the BA graphs and an average effect improve-

ment of 8.22 % in the real networks, which is significantly ahead of the other algorithms. Besides, for the other types of synthetic networks, FIGHTER has an average improvement of 0.42 % in the ER graphs and an average effect improvement of 0.21 % in the WS graphs, which exhibits strong generalization capability. The details are recorded in Figure Figs. 5(c) and Figure Figs. 6(c).

In the real world, the edges are more than nodes, resulting in most original graphs being sparse and line graphs being dense. For the task of attacking the connected subgraph with most nodes (GCC_{node}), FIGHTER can linearly and uniformly reduce the giant connected component (GCC)

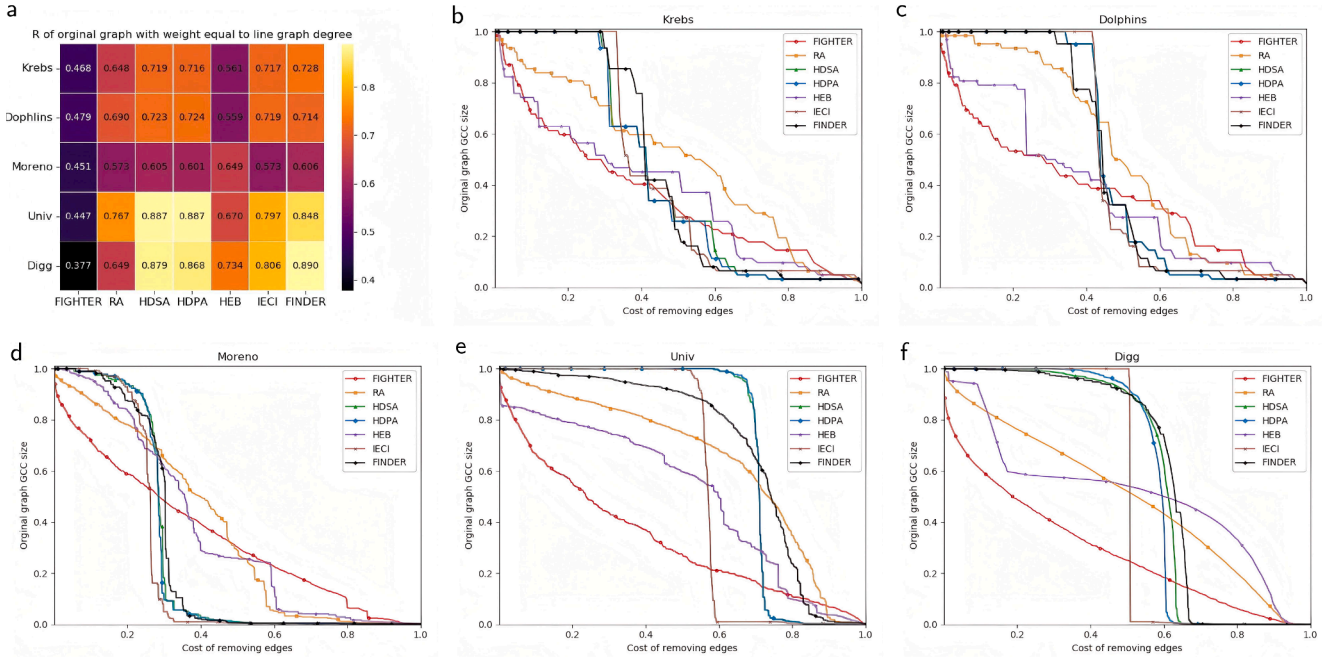


Fig. 9. The original graph's R value and decline curve under edge attack with weight equal to the line graph degree. The abscissa is the proportion of the cost of removing the edges, and the ordinate is the ratio of GCC_{node} size after the edges are removed to the GCC_{node} size at the beginning.

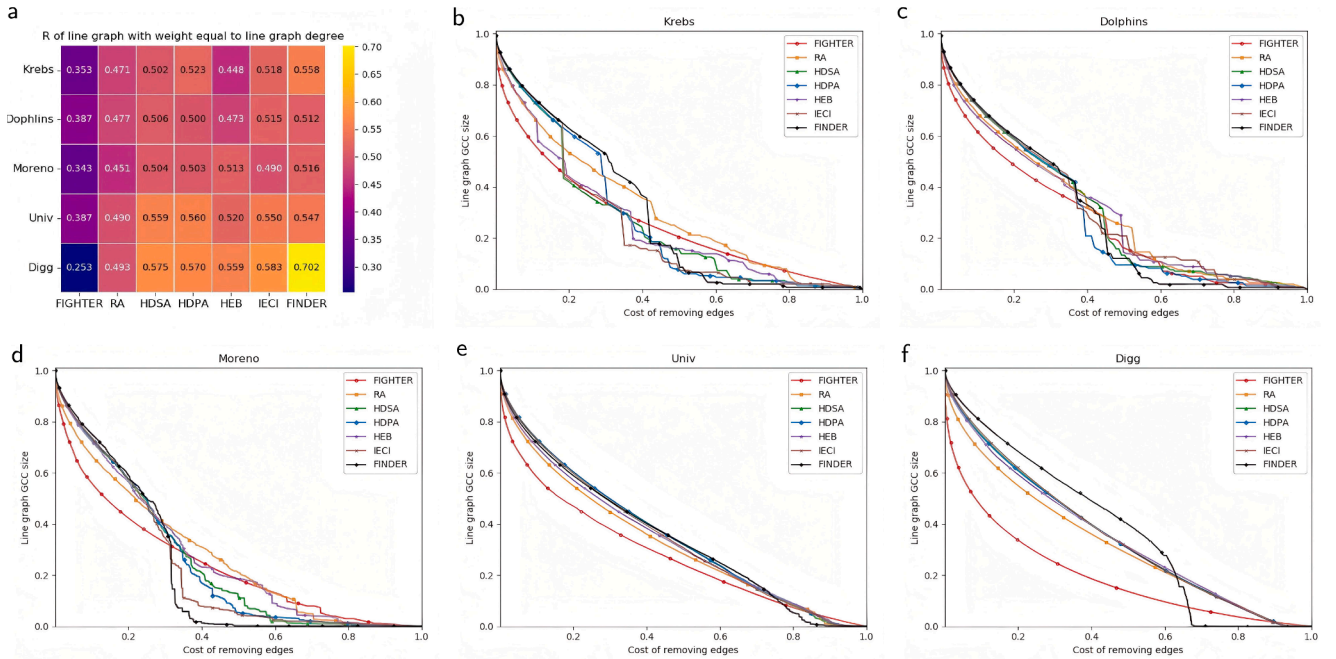


Fig. 10. The line graph's R value and decline curve under edge attack with weight equal to the line graph degree. The abscissa is the proportion of the cost of removing the edges, and the ordinate is the ratio of GCC_{edge} size after the edges are removed to the GCC_{edge} size at the beginning.

in the original graph, resulting in a smaller ANC area, quickly dismantling sparse graphs. For the task of attacking the connected subgraph with most edges (GCC_{edge}), FIGHTER can quickly identify the edges that correspond to key nodes in the line graph and complete dismantling dense graphs more quickly.

4.3.2. Heterogeneous-removal-cost attack scenario

For the task of attacking the connected subgraph with most nodes (GCC_{node}), results of attacking synthetic graphs' original graph and real networks' original graph are recorded in Figs. 4(b) and 9. Compared with the best results of other algorithms, FIGHTER has an average im-

provement of 34.22 % in the BA graphs and an average improvement of 25.37 % in the real networks, which is significantly ahead of the other algorithms. Besides, for the other types of synthetic networks, FIGHTER has an average improvement of 20.25 % in the ER graphs and an average effect improvement of 8.75 % in the WS graphs, which exhibits strong generalization capability. The details are recorded in Figure Figs. 5(b) and Figure Figs. 6(b).

Furthermore, for the task of attacking the connected subgraph with most edges (GCC_{edge}), results of attacking synthetic graphs' line graph and real networks' line graph are recorded in Figs. 4(d) and 10. Compared with the best results of other algorithms, FIGHTER has an

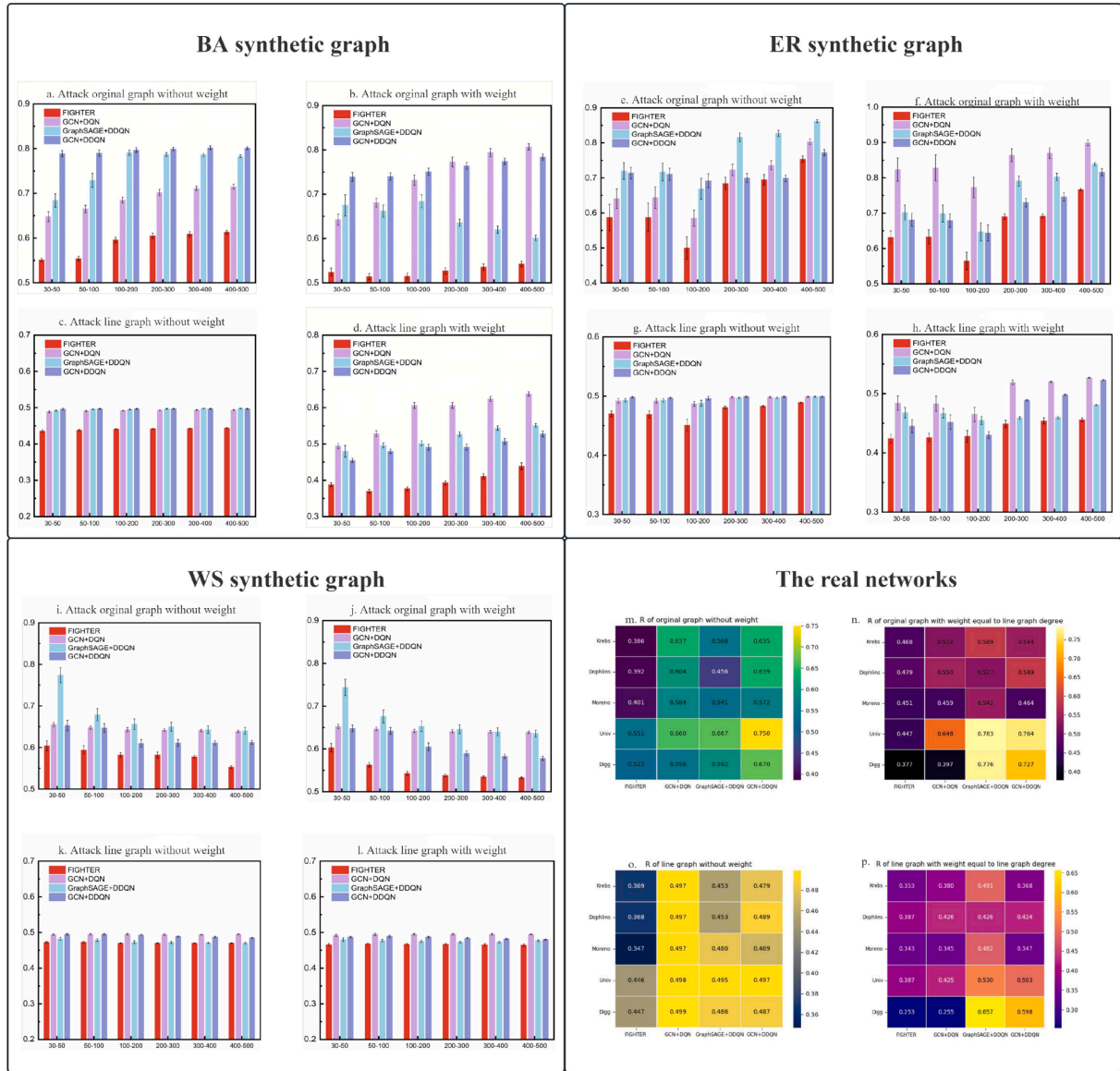


Fig. 11. The result of ablation study. Figure a to Figure l show experimental results of FIGHTER and three other deep neural network methods on BA synthetic networks, ER synthetic networks, and WS synthetic networks. Figure m to Figure p present results on real-world networks.

average improvement of 18.05% in the BA graphs and an average effect improvement of 26.46% in the real networks, significantly ahead of the other algorithms. Besides, for the other types of synthetic networks, FIGHTER has an average improvement of 9.65% in the ER graphs and an average effect improvement of 1.06% in the WS graphs, which exhibits strong generalization capability. The details are recorded in Figure Figs. 5(d) and Figure Figs. 6(d).

In most networks, few nodes have a high degree while most have a low degree, indicating that only a few nodes incur a higher attack cost. The line graph is converted from the original graph and has a similar degree distribution, so only a few edges in the original graph (the nodes in the line graph with a high degree) have a high attack cost. For the two ND tasks, attacking the connected subgraph with most nodes (GCC_{node}) and attacking the connected subgraph with most edges (GCC_{edge}), FIGHTER can maintain attack efficiency while avoid attacking edges with high-cost, significantly ahead of the other algorithms.

4.3.3. Ablation study

At first, we only used six strategies as the baseline, and our method achieved the best performance. However, to demonstrate FIGHTER's su-

periority, we replaced the GraphSAGE and DQN modules and added three baseline strategies:

- **GCN + DQN:** The Graph Convolutional Networks (GCN) (Kipf & Welling, 2017) is used as the method for graph embedding, and then combine DQN algorithm as the baseline (Mnih et al., 2013). Earlier research (Fan et al., 2019; Li, Chen, & Koltun, 2018; Manchanda et al., 2019; Zhu, Chen, Fan, Cheng, & Zhang, 2021) utilized GCN combined with DQN for combinatorial optimization problems, demonstrating favorable outcomes.
- **GraphSAGE + DDQN:** The DQN algorithm in the FIGHTER model is improved by using Double DQN instead of DQN (van Hasselt, Guez, & Silver, 2015), and anything else did not change in the model. Double DQN enhances the conventional DQN (Deep Q Network) to address the overestimation issue in its training phase. The problem arises because DQN employs the same network for both action selection and evaluation, leading to inflated target Q-values. To solve it, Double DQN divides the maximization of the target Q-value into two distinct steps: action selection and evaluation. The online and target networks handle these steps separately, reducing overestimation and boosting the algorithm's stability and effectiveness.

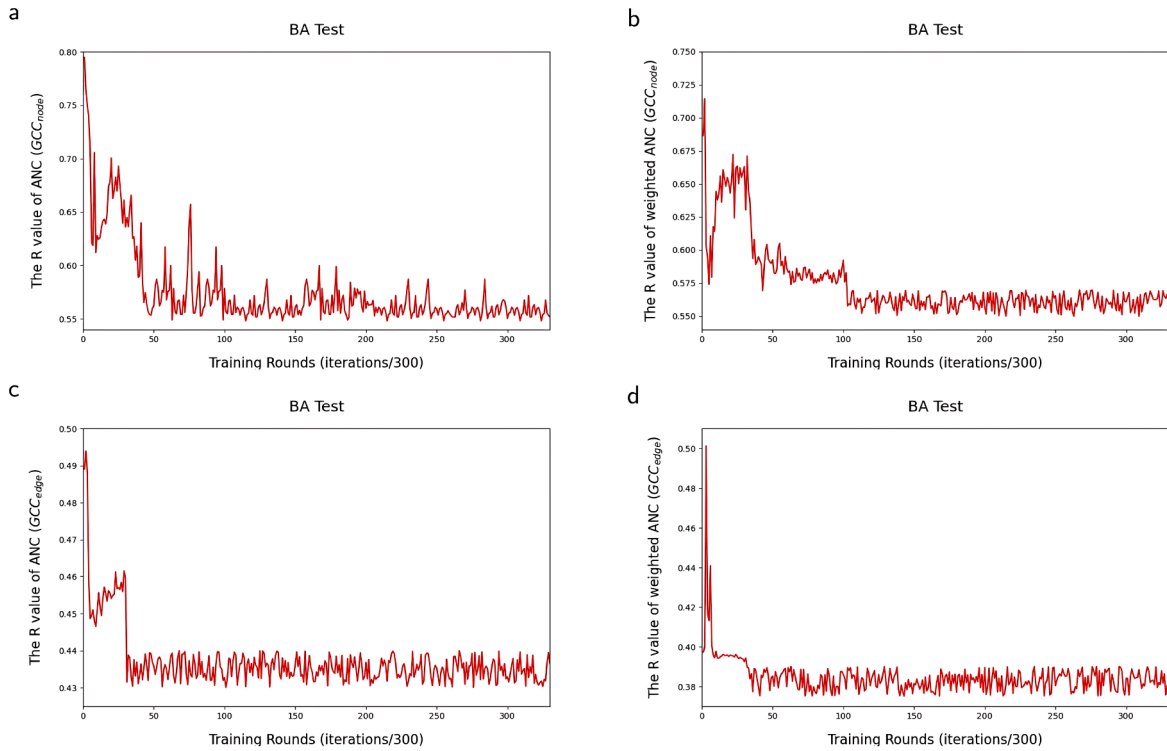


Fig. 12. The test results in BA synthetic graphs. During the training phase, the model is stored every 300 iterations. The parameters are recorded every 300 iterations and then tested in BA synthetic graphs.

- GCN+DDQN: The DQN algorithm in the FIGHTER model is improved by using Double DQN (van Hasselt et al., 2015), and the GraphSAGE model in the FIGHTER is improved by using GCN (Kipf & Welling, 2017).

We applied the trained stable model to the test set used in the paper, and recorded the results in Fig. 11. In the uniform-removal-cost attack scenario, for the task of attacking the connected subgraph with most nodes (GCC_{node}), FIGHTER has an average improvement of 14.53 % in the synthetic graphs and an average effect improvement of 18.61 % in the real networks, which is significantly ahead of the other algorithms. Furthermore, for the task of attacking the connected subgraph with most edges (GCC_{edge}), FIGHTER has an average improvement of 10.37 % in the synthetic graphs and an average effect improvement of 16.16 % in the real networks. In the heterogeneous-removal-cost attack scenario, for the task of attacking the connected subgraph with most nodes (GCC_{node}), FIGHTER has an average improvement of 18.42 % in the synthetic graphs and an average effect improvement of 13.74 % in the real networks, which is significantly ahead of the other algorithms. Furthermore, for the task of attacking the connected subgraph with most edges (GCC_{edge}), FIGHTER has an average improvement of 15.51 % in the synthetic graphs and an average effect improvement of 5.90 % in the real networks.

Moreover, methods like Prioritized Replay (Schaul, Quan, Antonoglou, & Silver, 2016), Dueling DQN (Wang, de Freitas, & Lanctot, 2015), Rainbow (Hessel et al., 2017) and soon on Bellemare, Dabney, and Munos (2017) have been proposed and are believed to enhance the model's convergence speed and performance. The graph representation ability and reinforcement learning algorithms are both influence model performance, and we will look into other DQN variants in the future.

Finally, we conducted statistical tests to evaluate significance. Independent Samples *t*-test is used for significance analysis. To simplify calculations, we averaged the means and standard deviations of 6 clusters and conducted significance tests for four network dismantling tasks across different network types. In the BA synthetic graphs and ER syn-

thetic graphs, compared with other methods (six baseline strategies and three ablation study methods), FIGHTER has P-values far below 0.01, which exhibits excellent performance. In the WS synthetic graphs, compared with HEB, FIGHTER has P-value of 0.016 for the task of attacking the connected subgraph with most edges (GCC_{edge}). In other cases, FIGHTER has P-values far below 0.01 compared with other methods in the WS synthetic graphs. A smaller P-value suggests a greater likelihood of differences between compared groups. Typically, $P < 0.05$ indicates significance, and $P < 0.01$ indicates high significance, meaning the probability that sample differences are due to random error is less than 0.05 or 0.01 and the P-value should not be equated with the magnitude of actual differences.

4.4. Time complexity

The value of the loss function gradually became stable. The *R* value of each network disintegration process during testing was calculated and averaged, which can be regarded as the performance of the model. The results obtained each time were drawn into the graph as shown in Fig. 12. It can be found that with the progress of training, the *R* obtained by FIGHTER on the test set became small and stable.

Let N represent the number of nodes, E represent the number of edges, $\langle k \rangle$ represent the average degree in the line graph and k represent the ratio of edges to nodes.

1. Line graph transformation

We transform the original graph into a line graph by using the adjacency list, with a time complexity of $O(N + E)$ (Peng et al., 2023).

2. Encoder

The time complexity of the Encoder procedure is $O(KE\langle k \rangle)$, where K is the step size of our aggregation in GraphSAGE, usually a smaller constant, $\langle k \rangle$ is the average degree in the line graph.

As shown in Fig. 13, $\langle k \rangle$ is approximately $4k-2$, which represents the degree sum value of the nodes at both ends of an edge minus the edge itself multiplied by two. The time complexity of encoding is approximately $O(kE)$.

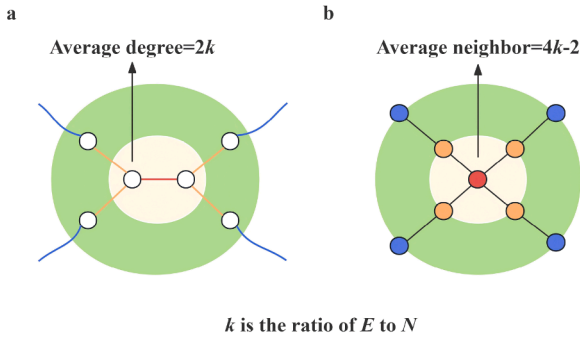


Fig. 13. Original graph convert to line graph.

Table 3

Time complexity of all algorithms and k is the ratio of edges to nodes.

Algorithm	Time complexity
RA (Albert et al., 2000)	$O(E)$
HEB (Girvan & Newman, 2002)	$O(N E + E \log E)$
HDSA (Holme & Kim, 2002)	$O(E^2 \log E)$
HDPA (Holme & Kim, 2002)	$O(E^2 \log E)$
IECI (Peng et al., 2023)	$O(N E + E^2 + E^2 \log E)$
FINDER (Fan et al., 2020c)	$O(N + k E + E \log E)$
FIGHTER	$O(N + k E + E \log E)$

3. Decoder

When the encoder process of the edges and the line graph are completed, the Q value of each edge is calculated and sorted, with a time complexity of $O(E \log E)$.

4. Greedy selection

In order to improve the efficiency of the algorithm, we adopt the strategy of batch edge selection and remove the top 1 % edges with the largest Q value each time.

The total time complexity of each step is $O(kE + E \log E)$. FIGHTER performs n adaptive steps, removing 1 % of the edges each time and we can consider n to be essentially constant.

Above all, the time complexity of the FIGHTER algorithm is approximately $O(N + kE + E \log E)$ and k equal to N in the worst case. Moreover, the line graph is applied to the FINDER algorithm, resulting in a sequence of edges rather than nodes for attack. Consequently, the time complexity of FINDER for edge attack changes from $O(N + E + E \log E)$ to $O(N + kE + E \log E)$.

Most real-world networks are sparse, with a small constant ratio of edges to nodes. The time complexity of each algorithm is shown in Table 3. The time complexity of FIGHTER is only slightly higher than RA, making it a highly efficient algorithm.

5. Conclusion

In the study, we introduced FIGHTER, an intelligent method that combines graph neural networks (GNN) and deep reinforcement learning (DRL) to identify key edges in the network and accelerate the network dismantling (ND) process.

FIGHTER has excellent effectiveness and stability, which can linearly and uniformly reduce the size of giant connected component (GCC) in the network, resulting in a smaller ANC area. As a deep reinforcement learning framework for edge attack, FIGHTER can efficiently attack a network with 10^6 edges in under 5 minutes, demonstrating strong generalization and excellent results. In terms of solving time, it is significantly better than other six algorithms in the large networks.

Additionally, line graph transformation is used for graph embedding, enhancing the effectiveness and generalization of FIGHTER. FIGHTER is an efficient and flexible model that applies to ND tasks like attack-

ing the subgraph with the most nodes or edges. The research focuses on undirected single-layer static network dismantling (ND), but real-world scenarios may involve heterogeneous networks, multi-layer networks and dynamic networks (such as the addition or deletion of nodes or edges). Our future work will explore edge-attack strategies in complex networks. We aim to enhance the FIGHTER algorithm for network disintegration in heterogeneous networks, multilayer networks, and dynamic networks, addressing current limitations in the field.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Fengwei Guo: Data curation, Software, Writing – original draft; **Chao Chen:** Conceptualization, Methodology; **Li Zeng:** Data curation, Writing – original draft; **Tianle Pu:** Software, Validation; **Feng Qing:** Visualization, Investigation; **Changjun Fan:** Supervision.

Funding

The study was supported by the [National Natural Science Foundation of China](#) (NSFC, [72421002](#), [62206303](#)), Science and Technology Innovation Program of Hunan Province (2023RC3009), Foundation Fund Program of National University of Defense Technology (JS24-05) and The Major Science and Technology Projects in Changsha, China (kq2301008).

References

- Albert, R., & Barabási, A. L. (2002). Statistical mechanics of complex networks. *Reviews of Modern Physics*, 74, 47–97.
- Albert, R., Jeong, H., & Albert-Laszlo, B. (2000). Error and attack tolerance of complex networks. *Nature*, (p. 378).
- Artme, O., Grassia, M., De Domenico, M., Gleeson, J. P., Makse, H. A., Mangioni, G., Perc, M., & Radicchi, F. (2024a). Robustness and resilience of complex networks. *Nature Reviews Physics*, (pp. 114–131).
- Artme, O., Grassia, M., De Domenico, M., Gleeson, J. P., Makse, H. A., Mangioni, G., Perc, M., & Radicchi, F. (2024b). Robustness and resilience of complex networks. *Nature Reviews Physics*, (pp. 114–131).
- Arulselvan, A., Commander, C. W., Eleftheriadou, L., & Pardalos, P. M. (2009). Detecting critical nodes in sparse graphs. *Computers and Operations Research*, (pp. 2193–2200).
- Bellemare, M. G., Dabney, W., & Munos, R. (2017). A distributional perspective on reinforcement learning. In *34th international conference on machine learning, ICML 2017*.
- Boccaletti, S., Latora, V., Moreno, Y., Chavez, M., & Hwang, D. U. (2006). Complex networks: Structure and dynamics. *Physics Reports*, 424(4), 175–308.
- Brandes, U. U.-k. (2008). On variants of shortest-path betweenness centrality and their generic computation. *Social Networks*, (pp. 136–145).
- Braunstein, A., Dall'Asta, L., Semerjian, G., & Zdeborová, L. (2016). Network dismantling. *Proceedings of the National Academy of Sciences Current Issue*, (p. 12368).
- Bröhl, T., & Lehnertz, K. (2022). A straightforward edge centrality concept derived from generalizing degree and strength. *Scientific Reports*, 12(1), 4407.
- Chen, L., Wang, L., Zeng, C., Liu, H., & Chen, J. (2023). Searching high-value edges attack sequence through deep reinforcement learning. *Knowledge-Based Systems*, 272, 110562.
- Cheng, X.-Q., Ren, F.-X., Shen, H.-W., Zhang, Z.-K., & Zhou, T. (2010). Bridgeness: A local index on edge significance in maintaining global connectivity(article). *Journal of Statistical Mechanics: Theory and Experiment*, (p. P10011).
- Dengcheng, Y., Wenxin, X., Yiwen, Z., Qiang, H., & Yun, Y. (2022). Hypernetwork dismantling via deep reinforcement learning. *IEEE Transactions on Network Science and Engineering*, 9(5), 3302–3315.
- Fan, C., Zeng, L., Ding, Y., Chen, M., Sun, Y., & Liu, Z. (2019). Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach. In *Proceedings of the 28th ACM international conference on information and knowledge management* (pp. 559–568).
- Fan, C., Zeng, L., Feng, Y., Cheng, G., Huang, J., & Liu, Z. (2020a). A novel learning-based approach for efficient dismantling of networks. *International Journal of Machine Learning and Cybernetics*, 11, 2101–2111.

- Fan, C., Zeng, L., Feng, Y., Xiu, B., Huang, J., & Liu, Z. (2020b). Revisiting the power of reinsertion for optimal targets of network attack. *Journal of Cloud Computing*, 9, 1–13.
- Fan, C., Zeng, L., Sun, Y., & Liu, Y.-Y. (2020c). Finding key players in complex networks through deep reinforcement learning. *Nature Machine Intelligence*, 2(6), 317–324.
- Feng, Y., Li, M., Zeng, C., & Liu, H. (2020a). Robustness of internet of battlefield things (iobt): a directed network perspective. *Entropy*, 22(10).
- Feng, Y., Ren, B., Zeng, C., Yang, Y., & Liu, H. (2020b). Enhancing edge attack strategy via an owa operator-based ensemble design in real-world networks. *Entropy*, 22(8).
- Feng, Y., Zeng, C., Li, M., & Liu, H. (2020c). Optimization of network disintegration strategy based on tabu-genetic hybrid search algorithm. In *2020 Chinese automation congress*, CAC (pp. 5108–5111).
- Freeman, L. C. (1977). A set of measures of centrality based on betweenness. *Sociometry*, (pp. 35–41).
- Girvan, M., & Newman, M. E. J. (2002). Girvan, m. newman, m. e. j. community structure in social and biological networks. *proc. natl acad. sci. USA* 99, 7821–7826. *Proceedings of the National Academy of Sciences*, (pp. 7821–7826).
- Grassia, M., & Mangioni, G. (2024). Edge dismantling with geometric reinforcement learning. *Complex Networks XV*, (pp. 195–202).
- Hagen, L., & Kahng, A. B. (1992). New spectral methods for ratio cut partitioning and clustering. *Computer-Aided Design of Integrated Circuits and Systems, IEEE Transactions on*, (pp. 1074–1085).
- van Hasselt, H., Guez, A., & Silver, D. (2015). Deep reinforcement learning with double q-learning. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence* (pp. 2094–2100).
- Hessel, M., Modayil, J., van Hasselt, H., Schaul, T., Ostrovski, G., Dabney, W., Horgan, D., Piot, B., Azar, M., & Silver, D. (2017). Rainbow: Combining improvements in deep reinforcement learning. In *32nd AAAI Conference on Artificial Intelligence* (pp. 3215–3222).
- Holme, P., & Kim, B. J. (2002). Attack vulnerability of complex networks. *Physical Review E, Statistical, Nonlinear, and Soft Matter Physics*, (p. 056109).
- J, X. X. Z. W. H., Zhang, P., & Han, H. (2017). Hybrid attack strategy based on edge weight. *Journal of Henan University of Science and Technology (Natural Science)*, 38(6), 38–43.
- Kanwar, K., Kaushal, S., Kumar, H., Gupta, G., & Khari, M. (2022). A new edge centrality measure to identify and rank critical edges pertaining to sir diffusion in complex networks. *Social Network Analysis and Mining*, 12(1), 49.
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *5th International Conference on Learning Representations*.
- Kunegis, J. (2013). Konect: The koblenz network collection. In *Www '13 companion: Proceedings of the 22nd international conference on world wide web*.
- Kuntz, I. D. (1992). Structure-based strategies for drug design and discovery. *Science*, (pp. 1078–1082).
- Lalou, M., Tahraoui, M. A., & Kheddouci, H. (2018). The critical node detection problem in networks: A survey. *Computer Science Review*, (pp. 92–117).
- Li, P. (2015). Robustness of network controllability based on topology-biased edge attacks. Ph.D. thesis. Xidian University.
- Li, Z., Chen, Q., & Koltun, V. (2018). Combinatorial optimization with graph convolutional networks and guided tree search. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems* (pp. 537–546).
- Liu, Y., Tang, M., Zhou, T., & Do, Y. (2015). Improving the accuracy of the k-shell method by removing redundant links: From a perspective of spreading dynamics. *Scientific Reports*, (p. 13172).
- Luxburg, U. V. (2007). A tutorial on spectral clustering. *Statistics and Computing*, (pp. 395–416).
- Manchanda, S., Mittal, A., Dhawan, A., Medya, S., Ranu, S., & Singh, A. K. (2019). Learning heuristics over large graphs via deep reinforcement learning. *arXiv Preprint arXiv:1903.03332*.
- Mao, J., Cao, L., Gao, C., Wang, H., Fan, H., Jin, D., & Li, Y. (2023). Detecting vulnerable nodes in urban infrastructure interdependent network. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 4617–4627).
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013). Playing atari with deep reinforcement learning.
- Morone, F., & Makse, H. (2015). Influence maximization in complex networks through optimal percolation. *Nature*, (pp. 65–68).
- Mugisha, S., & Zhou, H.-J. (2016). Identifying optimal targets of network attack by belief propagation. *Physical Review E*, (p. 012305).
- Newman, M. E. J. (2003). The structure and function of complex networks. *SIAM Review*, 45(2), 167–256.
- North, W. (2021). Attack-based network defense. *Cyber Defense Review*, 6(3), 129–138.
- Onnela, J.-P., Saramäki, J., Hyvönen, J., Szabó, G., Lazer, D., Kaski, K., Kertész, J., & Barabási, A.-L. (2007). Structure and tie strengths in mobile communication networks. *Proceedings of the National Academy of Sciences of the United States of America*, (pp. 7332–7336).
- Pastor-Satorras, R., & Vespignani, A. (2001). Epidemic spreading in scale-free networks. *Physical Review Letters*, (pp. 3200–3203).
- Peng, P., Fan, T., Ren, X.-L., & Linyuan, L. (2023). Unveiling explosive vulnerability of networks through edge collective behavior.
- Pournajar, M., Zaiser, M., & Moretti, P. (2022). Edge betweenness centrality as a failure predictor in network models of structurally disordered materials. *Scientific Reports*, 12(1), 11814.
- Qian, Y., Li, Y., Zhang, M., Ma, G., & Lu, F. (2017). Quantifying edge significance on maintaining global connectivity. *Scientific Reports*, (p. 45380).
- Ren, X.-L., Gleinig, N., Helbing, D., & Antulov-Fantulin, N. (2019). Generalized network dismantling. *Proceedings of the National Academy of Sciences of the United States of America*, (pp. 6554–6559).
- Ren, X.-L., Gleinig, N., Tolić, D., & Antulov-Fantulin, N. (2018). Underestimated cost of targeted attacks on complex networks. *Hindawi*, 2018.
- Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2016). Prioritized experience replay.
- Shao, S., Huang, X., Stanley, H. E., & Havlin, S. (2014). Percolation of localized attack on complex networks. *Computer Science*, (pp. 023–049).
- Singh, D. K., Nandi, S., Chakraborty, T., & Choudhury, P. (2022). Disintegrating constant communities in complex networks. *Journal of Computational Science*, 61, 101634.
- Tan, S.-Y., Wu, J., Linyuan, L., Li, M.-J., & Lu, X. (2016). Efficient network disintegration under incomplete information: The comic effect of link prediction. *Scientific Reports*, (p. 22916).
- Topirceanu, A., Udrescu, M., & Mărculescu, R. (2020). Complex networks antifragility under sustained edge attack-repair mechanisms. In *Proceedings of netsci-x 2020: Sixth international winter school and conference on network science*.
- Wang, C., Feng, Y., Zeng, C., & Liu, H. (2021). Optimal disintegration strategy in multiplex networks: from an edge-based attack perspective. In *2021 40th Chinese control conference, CCC* (pp. 788–794).
- Wang, H., Yang, R., Huang, K., & Xiao, X. (2023). Efficient and effective edge-wise graph representation learning. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining (KDD '23)* (p. 11). New York, NY, USA: ACM.
- Wang, Z., de Freitas, N., & Lancot, M. (2015). Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on International Conference on Machine Learning - Volume 48* (pp. 1995–2003).
- Xu, M. (2020). Researches on the Importance of Edges of Complex Networks. Ph.D. thesis. Shanxi University.
- Yiping, C., Gerald, P., Shlomo, H., Fredrik, L., & Stanley, H. E. (2008). Finding a better immunization strategy. *Physical Review Letters*, (p. 058701).
- Yu, E.-Y., Chen, D.-B., & Zhao, J. Y. (2018). Identifying critical edges in complex networks. *Scientific Reports*, 8(1), 14469.
- Zdeborová, L., Zhang, P., & Zhou, H.-J. (2016a). Fast and simple decycling and dismantling of networks. *Scientific Reports*, (pp. 37–44).
- Zdeborová, L., Zhang, P., & Zhou, H.-J. (2016b). Fast and simple decycling and dismantling of networks. *Scientific Reports*, (p. 37954).
- Zeng, C., Liu, H., Lu, L., Chen, J., & Zhou, Z. (2021). Shatter: Searching heterogeneous combat network attack sequences through network embedding and reinforcement learning. *IEEE Systems Journal*, 17(3), 1–12.
- Zeng, C., Lu, L., Liu, H., Chen, J., & Zhou, Z. (2022). Multiplex network disintegration strategy inference based on deep network representation learning. *Chaos (Woodbury, N.Y.)*, 32(4), 053109(1–15).
- Zhao, J., Sun, S., Shen, H., & Xia, C. (2021). An effective network repair strategy against both random and malicious edge attacks. In *2021 40th Chinese control conference, CCC*.
- Zhao, N., Li, J., Wang, J., Li, T., Yu, Y., & Zhou, T. (2020). Identifying significant edges via neighborhood information. *Physica A*, (p. 123877).
- Zhou, B., Lv, Y., Wang, J., Zhang, J., & Xuan, Q. (2022). Attacking the core structure of complex network. *IEEE Transactions on Computational Social Systems*, (pp. 1–15).
- Zhu, C., Chen, M., Fan, C., Cheng, G., & Zhang, Y. (2021). Learning from history: Modeling temporal knowledge graphs with sequential copy-generation networks. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 4732–4740). (35).